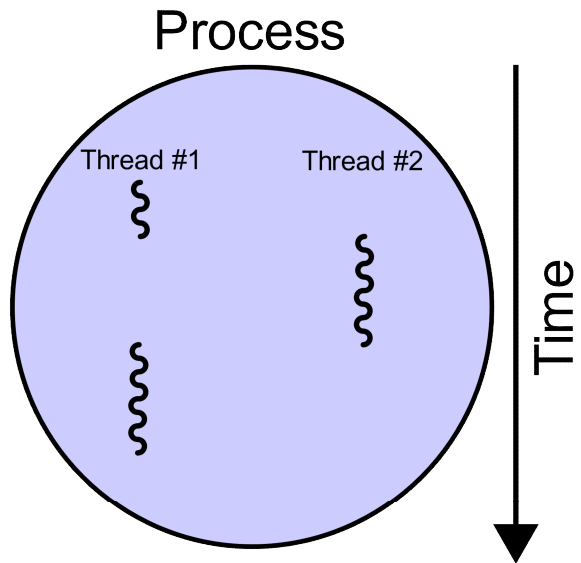


תרגול 5

חוטים (threads) בלינוקס
ספריית LinuxThreads
תכנות מקבילי באמצעות חוטים
מנגנוני סנכרון: מנעולים



TL;DR

- המעבדים של היום הם מרובי ליבות. איך אפשר לנצל אותם כדי לשפר ביצועים?
- לדוגמה, מיון מערך גדול ע"י: חלוקה לשניים, מיון כל חצי מערך בנפרד, ולבסוף מיזוג.

- **תהליכים** לא משתפים זיכרון, ולכן הם פחות מתאימים לתכנות מקבילי.
- **חוטים** (threads), לעומת זאת, פועלים במרחב זיכרון משותף.
- חוט הוא יחידת ביצוע עצמאית בתוך תהליך ("lightweight process").
- כל תהליך יכול להכיל מספר חוטים שירוצו במקביל.

- אבל שיתוף זיכרון יוצר גם בעיות, שאחת הנפוצות בהן היא: **היעדר אטומיות בגישה למשתנים משותפים**.
- נלמד איך להתגבר על הבעיה באמצעות מנעולים (mutexes).

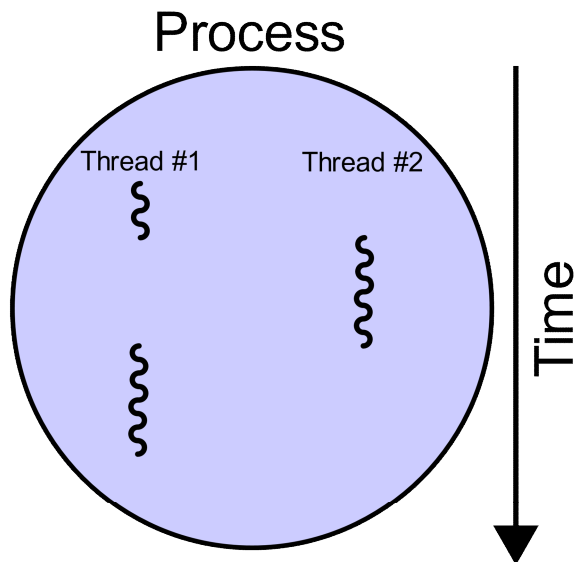
חוטאים (THREADS) בלינוקס

דוגמה: מיון מקבילי של מערך

```
void quick_sort(int a[], int length);

int* parallel_sort(int a[], int length) {
    int* b = (int*)malloc(length*sizeof(int));
    pid_t p = fork();
    if (p == 0) {
        quick_sort(a, N/2);
    } else {
        quick_sort(a + N/2, N/2);
        wait(NULL);
        // merge the two subarrays into b
    }
    return b;
}
```

מה הבעיה
במימוש המוצע?



חוטים (threads)

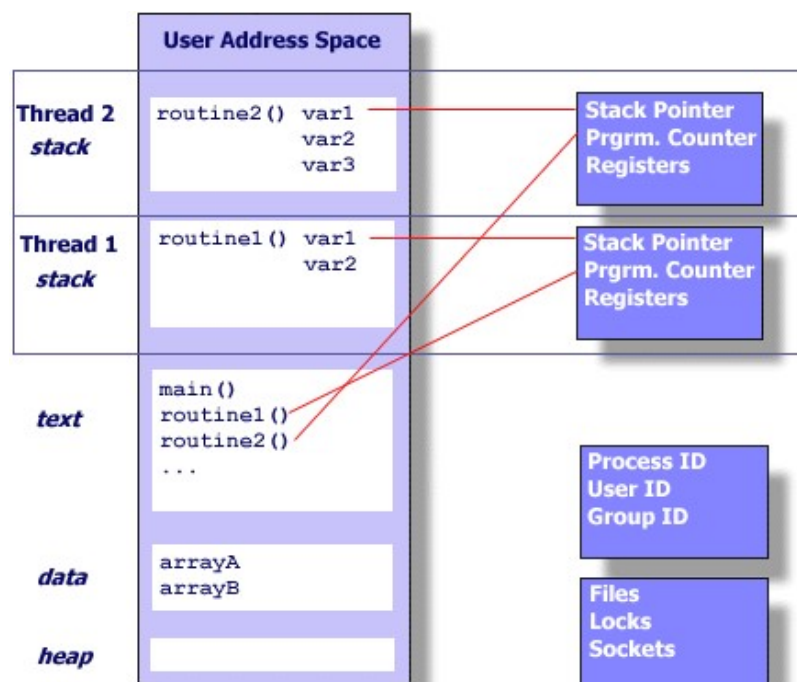
- חוט הוא יחידת ביצוע בתוך תהליך.
- באנגלית נקרא גם: lightweight process.
- בעברית נקרא גם: תהליכון, פתיל ריצה, נים.

• תהליך בלינוקס יכול לכלול מספר חוטים המשתפים ביניהם את כל משאבי התהליך: מרחב הזיכרון, גישה לקבצים והתקני חומרה, ועוד.

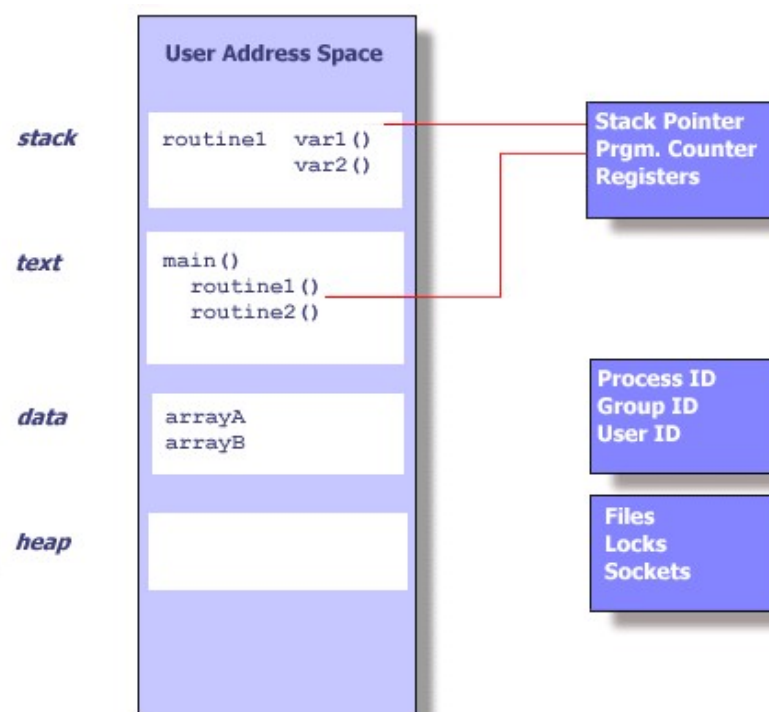
- למרות שחוט הוא רכיב של תהליך, כל חוט הוא יחידת ביצוע עצמאית שניתן להריץ על מעבד ללא קשר לשאר החוטים.
- כל חוט מהווה הקשר ביצוע נפרד – לכל חוט מחסנית ורגיסטרים משלו.
- ה-scheduler מזמן לריצה חוטים ולא תהליכים.

חוסים מול תהליכים

חוסים בתוך תהליך



תהליך



חוטים יכולים לשפר ביצועים

- במערכת מרובת מעבדים: כל חוט יפתור חלק מהמשימה של אותו תהליך, והחוטים ירוצו במקביל על מעבדים שונים.
 - דורש לפרק את הבעיה לתתי-בעיות בלתי תלויות.
- גם במחשב עם מעבד יחיד ניתן לשפר ביצועים באמצעות שימוש בריבוי חוטים – חוט אחד יכול לקרוא לקריאת מערכת חוסמת (למשל המתנה לנתונים מהרשת) וחוט אחר ימשיך בביצוע חלק אחר של התכנית.
- יצירת חוט לביצוע משימה זולה בהרבה מיצירת תהליך חדש לאותה מטרה, מפני שאינה כרוכה בהקצאת משאבים נוספים כגון זיכרון, גישה לחומרה וכו'.

תקשורת בין חוטים

- על-מנת לבצע את המשימה של התהליך, החוטים יצטרכו (לרוב) לתקשר על-מנת להחליף ביניהם מידע.
- דוגמה לאלגוריתם מקבילי שלא דורש שיתוף מידע בין חוטים: חיפוש מילה ספציפית במספר גדול של קבצים. כל חוט יקרא מספר קבצים וידפיס למסך את המופעים של המילה בקבצים שהוא בדק.
- אלגוריתמים כאלה נקראים embarrassingly parallel.
- התקשורת בין חוטים של אותו תהליך היא פשוטה ביותר: קריאה וכתיבה למשתנים משותפים.
- זהו גם חסרון: יש לתאם את הפעולות בין חוטים הניגשים לאותם משתנים על-מנת למנוע את שיבוש הנתונים.



מתי כדאי להשתמש בחוטים?

- יישומים שמתאימים במיוחד לריבוי חוטים:
 - תכניות המכילות מספר משימות בלתי תלויות, כגון הדפסת מסמך במקביל לעריכתו.
 - שרתים המטפלים במספר בקשות בו-זמנית: עדיף להפעיל חוט לכל בקשה מאשר תהליך לכל בקשה.
 - תכניות חישוב "כבדות" (למשל חיזוי מזג האוויר) הניתנות למיקבול יכולות לנצל את החומרה (ריבוי מעבדים) לשיפור הביצועים.
- יישומים שאינם מתאימים לריבוי חוטים:
 - תכניות קטנות ופשוטות: תקורה מיותרת (יצירת חוטים חדשים) מבלי לשפר ביצועים.
 - יישומים עתירי חישוב במערכת מעבד יחיד: תקורה מיותרת (החלפות הקשר) מבלי לשפר ביצועים.

ספריית LINUXTHREADS

שיעור היסטוריה

- בשנת 1995 הוצג **תקן pthreads** (POSIX Threads), המגדיר אוסף טיפוסים ופונקציות המאפשרים עבודה עם חוטים במערכות Unix.
 - התמיכה בחוטים בלינוקס שואפת להתאים לתקן זה.
 - מוגדר בקובץ `/usr/include/pthread.h`.
- בשנת 1996 הוצג המימוש הראשון של pthreads – **ספריית LinuxThreads**.
 - LinuxThreads היא ספריית משתמש (כמו libc), כלומר אינה חלק מקוד הגרעין.
 - LinuxThreads מתבססת על קריאת המערכת `clone()` כדי לממש חוטים ברמת הגרעין (kernel-level threads), כלומר חוטים המנוהלים ומתוזמנים ע"י גרעין מערכת ההפעלה.
- LinuxThreads מממשת באופן חלקי בלבד את תקן pthreads.
 - לדוגמה: `getpid()` מחזירה PID שונה לכל חוט.
 - כמו כן, המימוש בעייתי מבחינת ביצועים – לא סקאלאבילי.
 - למרות זאת, אנחנו נלמד על המימוש הקיים לפי LinuxThreads.

שיעור היסטוריה

- בשנת 2003 הוצגה **ספריית NPTL** (Native POSIX Thread Library), אשר מממשת בצורה מלאה את תקן pthreads.
 - גם NPTL היא ספריית משתמש (כלומר לא חלק מקוד הגרעין).
 - פותחה ע"י צוותים של IBM ו-Red Hat.
 - נכללה בהפצות Red Hat החל מגרסה 9.0.
- NPTL נשענת על תמיכה בגרעין לינוקס שקיימת החל מגרסה 2.6.
 - משיגה שיפור ניכר בביצועים ביחס ל-LinuxThreads.
- בשנת 2003 NPTL שולבה כחלק אינטגרלי מספריית GNU libc (שמקושרת אוטומטית לכל אפליקציות C שמקומפלת ע"י gcc).
 - כלומר היום pthreads בלינוקס היא "מילה נרדפת" ל-NPTL.
 - בשיעורי הבית תשתמשו ב-NPTL (כי תעבדו על השרת t2).

דוגמת קוד ליצירת חוטים

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void* f(void* arg) {
    printf("PID = %d,\npthread_id = %ld\n",
        getpid(), pthread_self());
    return NULL;
}

int main() {
    printf("Primary thread\n");
    f(NULL);

    pthread_t p;
    printf("New thread\n");
    pthread_create(&p, NULL, f, NULL);
    pthread_join(p, NULL);

    return 0;
}
```

שימוש ב-LinuxThreads

- כדי להשתמש בספריית LinuxThreads יש:
 - להוסיף קובץ header בתחילת קוד C:

```
#include <pthread.h>
```

- להוסיף את הדגל pthread- במהלך ההידור:

```
gcc myprog.c -pthread -o myprog
```

- הדגל מגדיר מספר macros ומקשר את התכנית עם הספרייה הנחוצה.

• הספרייה LinuxThreads:

- מממשת (באופן חלקי) את הממשק של POSIX Threads לעבודה עם חוטים.
- משנה את פעולתן של מספר קריאות מערכת הקשורות לעבודה עם תהליכים (למשל fork), כפי שנראה בהמשך.
- הספרייה לא משנה את המימוש של קריאות המערכת בתוך הגרעין!
- היא רק מחליפה את פונקציות המעטפת שקוראות לקריאות המערכת (במקום מימוש ברירת המחדל של libc).

יצירת חוט חדש

```
int pthread_create(pthread_t *thread,  
    pthread_attr_t *attr,  
    void* (*start_routine)(void*),  
    void *arg);
```

- פעולה: יוצרת חוט חדש המתבצע במקביל לחוט הקורא בתוך אותו תהליך. החוט החדש מתחיל לבצע את הפונקציה המופיעה בפרמטר start_routine ומת בסיום ביצוע הפונקציה.
- ערך מוחזר:
 - 0 במקרה של הצלחה. כמו כן, מזהה החוט החדש נכתב למשתנה המוצבע ע"י thread.
 - ערך שגיאה שלילי במקרה של כישלון.

יצירת חוט חדש

• פרמטרים:

- thread – מצביע למקום בו יאוחסן מזהה החוט החדש במקרה של סיום הפונקציה בהצלחה.
- attr – מאפיינים המתארים את תכונות החוט החדש, כגון האם החוט הוא חוט מערכת או חוט משתמש, האם ניתן לבצע לו join, כלומר להמתין לסיומו, וכו'. בד"כ נספק ערך NULL המציין חוט מערכת שניתן להמתין לסיומו.
- start_routine – מצביע לפונקציה שתהווה את קוד החוט. הערך המוחזר מפונקציה זו במקרה של סיומה הטבעי הינו ערך הסיום של החוט.
- arg – פרמטר שיסופק לפונקציה עם הפעלתה.

סיום חוט

```
void pthread_exit(void *retval);
```

- פעולה: מסיימת את פעולת החוט הקורא. ערך הסיום יוחזר לחוט שימתין לסיום חוט זה.
- פרמטרים:
 - `retval` – ערך סיום (בדומה לזה של `exit()`).
 - ערך מוחזר: אין.
- סיום פעולת החוט הראשי ע"י `pthread_exit()` אינו מסיים את כל החוטים בתהליך. במצב זה, התהליך נותר ללא חוט ראשי.
- אם לחוט כלשהו יש בנים (תהליכים), הם הופכים להיות בנים של חוט אחר בקבוצת האב לאחר מותו.
- רק אם כל החוטים של תהליך האב הסתיימו, התהליך `init` יירש את תהליך הבן.

הריגת חוט

```
int pthread_cancel(pthread_t thread);
```

- פעולה: מסיימת את ביצוע החוט המזוהה ע"י thread.
- ערך סיום הביצוע של החוט שנהרג יהיה PTHREAD_CANCELED.

- פרמטרים:
- thread – מזהה החוט המיועד לסיום.

- ערך מוחזר:
- 0 במקרה של הצלחה.
- ערך שגיאה שלילי במקרה של כישלון.

קבלת מזהה החוט

```
pthread_t pthread_self();
```

- פעולה: מחזירה לחוט הקורא את המזהה של עצמו. מזהה זה הוא פנימי לספרייה LinuxThreads ואינו קשור ל-PID של החוט.
- פרמטרים: אין.
- ערך מוחזר: מזהה החוט.

המתנה לסיום חוט

```
int pthread_join(pthread_t thread,  
void **thread_return);
```

- פעולה: גורמת לחוט הקורא להמתין לסיום החוט המזוהה ע"י `.thread`.
 - ניתן להמתין על סיום אותו חוט פעם אחת לכל היותר.
 - ביצוע `pthread_join` על אותו חוט יותר מפעם אחת ייכשל.
 - כל חוט יכול להמתין לסיום כל חוט אחר באותו תהליך.
 - ההמתנה על סיום החוט משחררת את מידע הניהול של החוט ברמת `LinuxThreads` וברמת הגרעין.

המתנה לסיום חוט

- פרמטרים:

- thread – מזהה החוט שממתינים לסיומו.
 - לא ניתן להמתין ל"סיום חוט כלשהו" בדומה ל-wait().
 - thread_return – מצביע למקום בו יאוחסן ערך הסיום של החוט עבורו ממתינים.
 - ניתן לציין NULL כדי להתעלם מערך הסיום.
-
- ערך מוחזר: 0 במקרה של הצלחה, וערך שונה מ-0 במקרה כישלון. כמו כן, במקרה של הצלחה ערך הסיום נכתב למשתנה המוצבע ע"י thread_return (אם אינו NULL).